

# Diseño e Implementación de Controlador PID Digital en ESP32 (ESP-IDF)

Ingeniería de Control y Sistemas Embebidos

8 de diciembre de 2025

## 1. Descripción del Sistema

El objetivo es regular la tensión de salida de un convertidor Buck ( $V_{out}$ ) a un valor de referencia de 6V, partiendo de una fuente de 12V. El sistema de control se implementa en un microcontrolador ESP32 utilizando el framework nativo ESP-IDF.

### 1.1. Parámetros de la Planta y Actuador

- **Entrada ( $V_{in}$ ):** 12V DC.
- **Referencia ( $V_{ref}$ ):** 6V.
- **PWM:** Frecuencia de 50kHz, resolución de 10 bits.
- **Sensor:** Divisor de tensión resistivo ( $39k\Omega$  /  $10k\Omega$ ).

## 2. Diseño de la Interfaz de Hardware

Para adecuar la tensión de salida del Buck (0 – 12V) al rango dinámico del ADC del ESP32, se emplea un divisor de tensión. Basándonos en el cálculo técnico provisto:

### 2.1. Selección de Resistencias

Se han seleccionado los siguientes valores según especificación docente:

- $R_1 = 39\text{ k}\Omega$  (Resistencia superior)[cite: 6].
- $R_2 = 10\text{ k}\Omega$  (Resistencia inferior a GND)[cite: 7].

La resistencia equivalente del divisor es  $R_{eq} = 49\text{ k}\Omega$ [cite: 19].

## 2.2. Validación de Niveles de Tensión

El factor de escala del divisor ( $K_{div}$ ) se define como:

$$K_{div} = \frac{R_2}{R_1 + R_2} = \frac{10}{49} \approx 0,2041 \quad (1)$$

Para el caso crítico donde  $V_{in.buck} = 12V$  (ciclo de trabajo al 100 %):

$$V_{ADC(max)} = 12V \times 0,2041 \approx 2,45V \quad (2)$$

Este valor coincide con el cálculo teórico de 2,45V, lo cual es seguro para el ESP32 configurado con atenuación de 11dB (rango lineal hasta aprox. 2,6V).

## 2.3. Consideraciones de Potencia

La potencia disipada total en el divisor es de aproximadamente 2,94mW[cite: 44]. Dado que las resistencias de 1/4W (250mW) soportan esta carga con un amplio margen de seguridad[cite: 52], el diseño es robusto.

# 3. Diseño del Controlador PID

## 3.1. Función de Transferencia

La planta requiere el siguiente controlador en tiempo continuo:

$$C(s) = \frac{0,00243s^2 + 3s + 450}{0,0001s^2 + s} \quad (3)$$

Esta ecuación corresponde a la estructura PID con filtro derivativo:

$$C(s) = K_p + \frac{K_i}{s} + \frac{K_d s}{T_f s + 1} \quad (4)$$

**Parámetros extraídos:**

$$K_p = 2,955$$

$$K_i = 450$$

$$K_d = 0,0021345$$

$$T_f = 0,0001s \quad (Frecuencia de corte \approx 1,6kHz)$$

## 3.2. Discretización (Método Tustin)

Para una frecuencia de muestreo sincronizada con el PWM ( $F_s = 50kHz$ ), el tiempo de muestreo es  $T_s = 20\mu s$ .

Las ecuaciones en diferencias para la implementación digital son:

### 1. Término Integral ( $I_k$ ):

$$I_k = I_{k-1} + \alpha(e_k + e_{k-1}) \quad \text{donde } \alpha = \frac{K_i T_s}{2}$$

## 2. Término Derivativo ( $D_k$ ):

$$D_k = \frac{\beta_{num}(e_k - e_{k-1}) + \gamma D_{k-1}}{\beta_{den}}$$

Donde:

$$\beta_{den} = 2T_f + T_s, \quad \beta_{num} = 2K_d, \quad \gamma = 2T_f - T_s$$

## 3. Salida de Control ( $u_k$ ):

$$u_k = K_p e_k + I_k + D_k$$

## 4. Implementación en ESP-IDF (C Puro)

A continuación se presenta el código fuente para el ESP32. Se utiliza el driver `ledc` para la generación de PWM y `adc_oneshot` con calibración para la lectura precisa del voltaje.

```
1 #include <stdio.h>
2 #include "freertos/FreeRTOS.h"
3 #include "freertos/task.h"
4 #include "driver/ledc.h"
5 #include "esp_adc/adc_oneshot.h"
6 #include "esp_adc/adc_cali.h"
7 #include "esp_adc/adc_cali_scheme.h"
8 #include "esp_timer.h"
9
10 // --- Definiciones de Hardware ---
11 #define PWM_GPIO 18
12 #define ADC_CHANNEL ADC_CHANNEL_6 // GPIO 34 en ESP32 WROVER/DevKit
13 #define ADC_ATTEN ADC_ATTEN_DB_11 // Rango hasta ~2.5V (Perfecto
    para 2.45V max)
14 #define ADC_WIDTH ADC_BITWIDTH_12
15
16 // --- Constantes del Sistema ---
17 #define PWM_FREQ_HZ 50000
18 #define PWM_RES_BITS LEDC_TIMER_10_BIT
19 #define PWM_MAX_DUTY 1023 // (2^10 - 1)
20 #define V_REF_TARGET 6.0f // Objetivo 6V
21
22 // Divisor resistivo: R1=39k, R2=10k -> Factor = 10/49
23 #define DIVIDER_RATIO (10.0f / 49.0f) // aprox 0.20408
24
25 // --- Variables PID y Coeficientes Tustin ---
26 // Ts = 20us (0.00002s)
27 float Kp = 2.955f;
28 // Alpha = Ki * Ts / 2 = 450 * 0.00002 / 2
29 const float alpha = 0.0045f;
30 // Coeficientes Derivativos (Tf=0.0001, Ts=0.00002)
31 // beta_den = 2*Tf + Ts = 0.00022
32 const float beta_den = 0.00022f;
33 // beta_num = 2*Kd = 2 * 0.0021345
34 const float beta_num = 0.004269f;
35 // gamma = 2*Tf - Ts = 0.00018
36 const float gamma_val = 0.00018f;
37
38 // Estados del Controlador
```

```

39 float integral = 0.0f;
40 float prev_error = 0.0f;
41 float prev_derivative = 0.0f;
42
43 // --- Manejadores Globales ---
44 adc_oneshot_unit_handle_t adc1_handle;
45 adc_cali_handle_t adc_cali_handle = NULL;
46
47 // Inicializacion del PWM (LEDC)
48 void init_pwm() {
49     ledc_timer_config_t ledc_timer = {
50         .speed_mode      = LEDC_HIGH_SPEED_MODE,
51         .timer_num       = LEDC_TIMER_0,
52         .duty_resolution  = PWM_RES_BITS,
53         .freq_hz         = PWM_FREQ_HZ,
54         .clk_cfg         = LEDC_AUTO_CLK
55     };
56     ledc_timer_config(&ledc_timer);
57
58     ledc_channel_config_t ledc_channel = {
59         .speed_mode      = LEDC_HIGH_SPEED_MODE,
60         .channel         = LEDC_CHANNEL_0,
61         .timer_sel       = LEDC_TIMER_0,
62         .intr_type       = LEDC_INTR_DISABLE,
63         .gpio_num        = PWM_GPIO,
64         .duty            = 0,
65         .hpoint          = 0
66     };
67     ledc_channel_config(&ledc_channel);
68 }
69
70 // Inicializacion del ADC con Calibracion
71 void init_adc() {
72     adc_oneshot_unit_init_cfg_t init_config1 = {
73         .unit_id = ADC_UNIT_1,
74     };
75     adc_oneshot_new_unit(&init_config1, &adc1_handle);
76
77     adc_oneshot_chan_cfg_t config = {
78         .bitwidth = ADC_WIDTH,
79         .atten = ADC_ATTEN,
80     };
81     adc_oneshot_config_channel(adc1_handle, ADC_CHANNEL, &config);
82
83     // Calibracion por curva (Linear fitting)
84     adc_cali_curve_fitting_config_t cali_config = {
85         .unit_id = ADC_UNIT_1,
86         .atten = ADC_ATTEN,
87         .bitwidth = ADC_WIDTH,
88     };
89     adc_cali_create_scheme_curve_fitting(&cali_config, &adc_cali_handle)
90 ;
91 }
92
93 // Bucle Principal de Control
94 void app_main(void) {
95     init_pwm();
96     init_adc();

```

```

96
97     int adc_raw;
98     int voltage_mv;
99
100    // Bucle infinito
101    // Nota: Para estricto tiempo real a 50kHz, esto deberia ir
102    // en una interrupcion de timer, pero aqui se demuestra la logica.
103    while (1) {
104        int64_t start_time = esp_timer_get_time();
105
106        // 1. Lectura y conversion
107        adc_oneshot_read(adc1_handle, ADC_CHANNEL, &adc_raw);
108        // Convierte raw a milivoltios reales calibrados
109        adc_cali_raw_to_voltage(adc_cali_handle, adc_raw, &voltage_mv);
110
111        // Convertir a voltaje de salida real del Buck
112        //  $V_{out} = V_{adc} / K_{div}$ 
113        float v_out_measured = (voltage_mv / 1000.0f) / DIVIDER_RATIO;
114
115        // 2. Calculo de Error
116        float error = V_REF_TARGET - v_out_measured;
117
118        // 3. PID (Tustin)
119        // Proporcional
120        float P = Kp * error;
121
122        // Integral
123        integral += alpha * (error + prev_error);
124        // Anti-windup (Saturacion del integrador)
125        if (integral > 10.0f) integral = 10.0f;
126        if (integral < -10.0f) integral = -10.0f;
127
128        // Derivativo
129        float D = (beta_num * (error - prev_error) + gamma_val *
prev_derivative) / beta_den;
130
131        // Salida total (u es duty cycle teorico 0.0 - 1.0)
132        float u = P + integral + D;
133
134        // Saturacion del actuador
135        if (u > 0.90f) u = 0.90f; // Max 90% por seguridad
136        if (u < 0.0f) u = 0.0f;
137
138        // 4. Actuacion
139        uint32_t duty = (uint32_t)(u * PWM_MAX_DUTY);
140        ledc_set_duty(LEDCH_HIGH_SPEED_MODE, LEDC_CHANNEL_0, duty);
141        ledc_update_duty(LEDCH_HIGH_SPEED_MODE, LEDC_CHANNEL_0);
142
143        // 5. Actualizacion de estados
144        prev_error = error;
145        prev_derivative = D;
146
147        // Espera activa o delay para ajustar a 50kHz (aprox)
148        // En produccion usar GPTimer ISR.
149        // Simulamos un delay minimo para evitar Watchdog si no hay OS
tick
150        vTaskDelay(1); // Esto es 1ms (1kHz), para 50kHz se requiere
Timer ISR

```

```
151 }
152 }
```

Listing 1: Firmware de Control PID en ESP-IDF

## 5. Consideraciones Finales

- **Precisión del ADC:** Al utilizar `adc_cali`, obtenemos el voltaje real en el pin compensando la no linealidad del ESP32.
- **Frecuencia de Lazo:** El código en `while(1)` con `vTaskDelay` correrá a 1kHz (limitado por FreeRTOS tick). Para alcanzar los **50kHz** reales requeridos por el diseño, toda la lógica dentro del bucle `while` debe moverse a una **Interrupción de Timer (GPTimer)** o utilizar un task de alta prioridad sin bloqueos y con `esp_rom_delay_us`.